Name: G Venkata Ganesh
Reg.NO: 192425381

In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Example 1:

Input: s = "abbxxxxzzy"

Output: [[3,6]]

Explanation: "xxxx" is the only large group with start index 3 and end index 6.

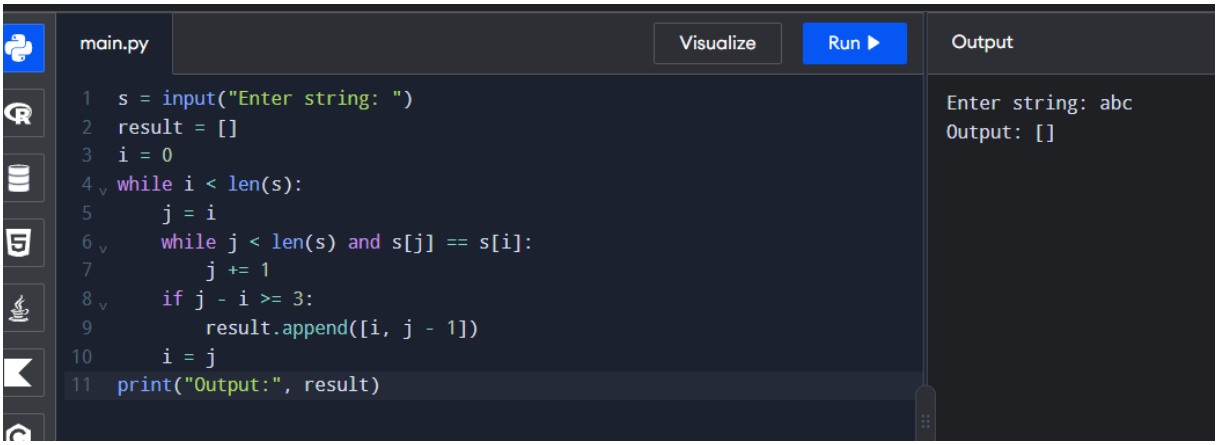


Example 2:

Input: s = "abc"

Output: []

Explanation: We have groups "a", "b", and "c", none of which are large groups.



4. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

   Any live cell with fewer than two live neighbors dies as if caused by under-population.

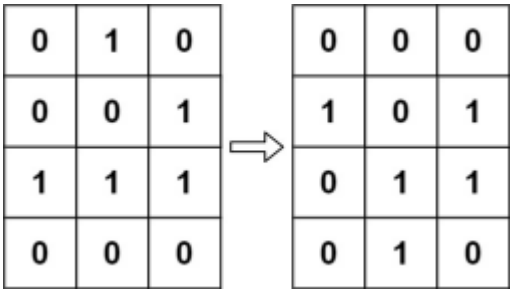
   Any live cell with two or three live neighbors lives on to the next generation.

   Any live cell with more than three live neighbors dies, as if by over-population.

   Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

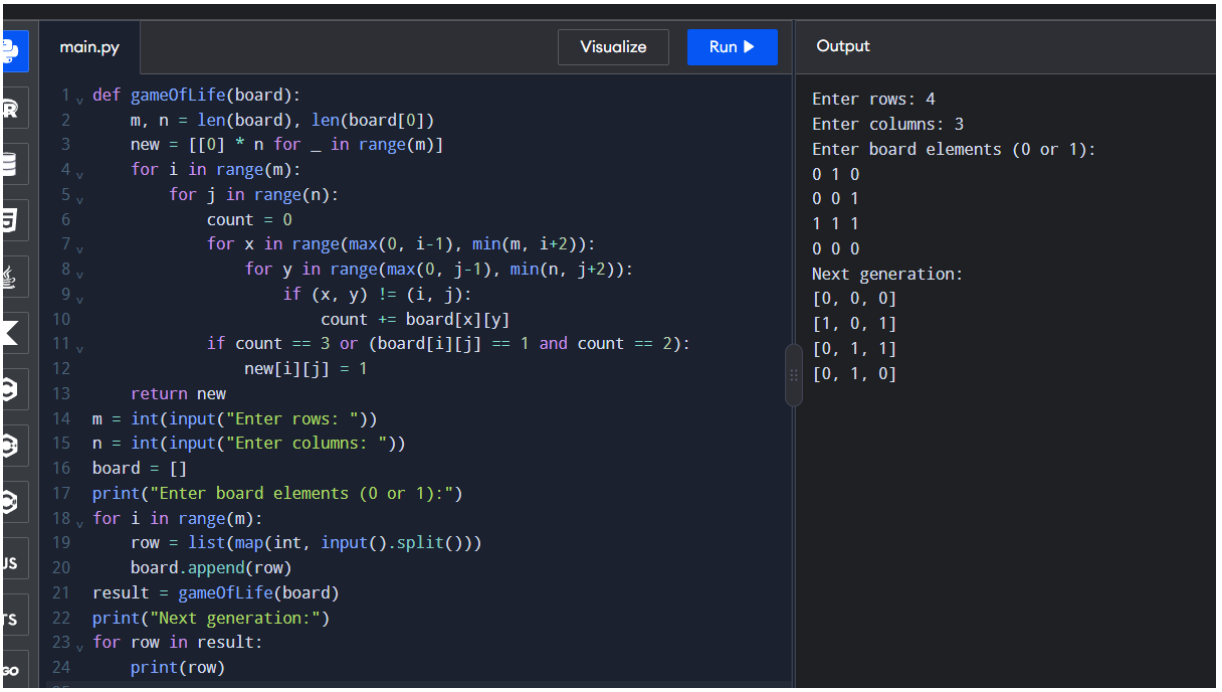
   The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the m x n grid board, return *the next state*.

   Example 1:

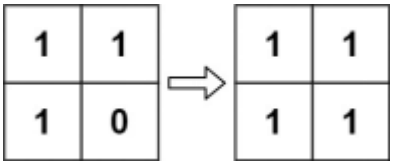


Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]

Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]



Example 2:



Input: board = [[1,1],[1,0]]

Output: [[1,1],[1,1]]

main.py

Visualize

Run ▶

```python
def gameOfLife(board):
    m, n = len(board), len(board[0])
    new = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            count = 0
            for x in range(max(0, i-1), min(m, i+2)):
                for y in range(max(0, j-1), min(n, j+2)):
                    if (x, y) != (i, j):
                        count += board[x][y]
            if count == 3 or (board[i][j] == 1 and count == 2):
                new[i][j] = 1
    return new
m = int(input("Enter rows: "))
n = int(input("Enter columns: "))
board = []
print("Enter board elements (0 or 1):")
for i in range(m):
    row = list(map(int, input().split()))
    board.append(row)
result = gameOfLife(board)
print("Next generation:")
for row in result:
    print(row)
```

Output

```
Enter rows: 2
Enter columns: 2
Enter board elements (0 or 1):
1 0
1 1
Next generation:
[1, 1]
[1, 1]
```

5. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.



Now after pouring some non-negative integer cups of champagne, return how full the jth glass in the ith row is (both i and j are 0-indexed.)

Example 1:

Input: poured = 1, query_row = 1, query_glass = 1

Output: 0.00000

Explanation: We poured 1 cup of champange to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty.



Example 2:

Input: poured = 2, query_row = 1, query_glass = 1

Output: 0.50000

Explanation: We poured 2 cups of champange to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champange.

